

PROJECT CONSENSUS 3D

System & Game Implementation Design

Version 2.1 • V1 Implementation Baseline • 12 September 2026

A deterministic, asynchronous social prediction game in which player votes generate a three-digit collective outcome and predictions compete against that outcome using independent per-digit distance.

Decision	V1 baseline
Round lifecycle	24h entry • 1h blackout • T+24 seal • T+30 reveal
Outcome engine	D'Hondt allocation across three digit positions
Prediction scoring	Sum of absolute distances across the three digits
Winner model	All players achieving minimum digit distance are winners
Persistence	PostgreSQL authoritative; Redis only for recoverable/derived workloads
Coordination	External/social coordination is permitted gameplay
Integrity objective	Durable, deterministic, auditable—not government-grade identity verification

Implementation principle: correctness first. Any Redis, cache, worker, or UI failure must not alter or silently lose authoritative player state.

Prepared as the implementation-ready consolidation of the approved v2.0 design and subsequent game/system decisions.

1. Product Definition & Core Rules

Each player controls two independent choices: a three-digit prediction (000-999) and one vote digit (0-9). Votes determine the collective three-digit result; predictions are evaluated only after that result exists.

1.1 Round lifecycle

Phase	Time	Behavior
Open	T+0 → T+23h	Drafts editable; manual lock-in; hourly public metrics.
Blackout	T+23h → T+24h	Live leaderboard/vote distribution hidden; submissions remain possible.
Sealing	At T+24h	No new mutations; valid remaining drafts automatically commit.
Resolution	T+24h → T+30h	Immutable tally, D'Hondt, winner-distance computation, audit generation.
Reveal	T+30h	Winning number, prediction histogram, player result and audit information.

1.2 Submission states

DRAFT

mutable; durable; excluded from tally and winner calculation

COMMITTED

immutable; included in tally and winner calculation
assigned authoritative commit_sequence

ROUND SEALED

no further mutation is accepted

Manual lock-in and deadline auto-lock are the same logical transition: DRAFT → COMMITTED. The difference is only who initiates it.

1.3 Non-negotiable invariants

- One account may have at most one active round entry.
- A committed entry can never be edited, withdrawn, or replaced.
- Prediction and vote are independent; a vote need not appear in the prediction.
- Prediction frequency never changes the winning number or prediction distance.
- Vote choice affects only the collective outcome.
- All players at the minimum digit distance share the winning tier; there is no timestamp-based winner tie-break.

2. Game Mathematics & Resolution Engine

2.1 D'Hondt outcome generation

Let $C[d]$ be the validated committed vote count for digit $d \in \{0 \dots 9\}$. For each of three positions, maintain $s[d]$, the number of positions already allocated to d . Evaluate $Q(d) = C[d] / (s[d] + 1)$. Highest quotient wins; equal quotients are resolved permanently in favor of the lower digit.

```
counts[d] = committed_vote_count[d]
slots_won[d] = 0

for position in 1..3:
    q[d] = Fraction(counts[d], slots_won[d] + 1)
    winner = argmax(q[d], then lower d)
    slots_won[winner] += 1
    result[position] = winner
```

No special zero-vote exception is required. If every $C[d] = 0$, every quotient is zero and the permanent lower-digit tie-break selects 0 for every position, producing 000.

2.2 Exact arithmetic

Use rational/integer comparison rather than floating point. Compare a/b and c/d through cross multiplication or use Python Fraction in the reference implementation. The result must be identical across environments.

2.3 Examples

Votes	Position 1	Position 2	Position 3	Result
2:300, 7:95	2 (300)	2 (150)	2 (100 > 95)	222
1:100, 2:99	1 (100)	2 (99 > 50)	1 (50 > 49.5)	121
All digits: 0	0 (tie)	0 (tie)	0 (tie)	000

2.4 Prediction distance

For prediction $P = (P_1, P_2, P_3)$ and winning number $W = (W_1, W_2, W_3)$:

$$D(P, W) = |P_1 - W_1| + |P_2 - W_2| + |P_3 - W_3|$$

This is deliberately not numerical distance. Each position contributes equally. The minimum distance is the winning distance and every player achieving it is a winner.

Example: $W=527$. Predictions $527 \rightarrow 0$; $526 \rightarrow 1$; $517 \rightarrow 1$; $427 \rightarrow 1$; $529 \rightarrow 2$.

3. Winner Model, Rankings & Player Feedback

3.1 Winner set

```
min_distance = MIN(D(prediction, winning_number))
winners = all committed submissions where
    D(prediction, winning_number) == min_distance
```

There is no final single-player tie-break. If 14 players achieve distance 0, all 14 are exact winners. Submission timing is retained for audit/history but does not alter prediction rank.

3.2 Prediction histogram

The reveal presents the distribution of players by digit distance. It is descriptive only and is computed after the winning number is fixed; it cannot feed back into outcome resolution.

Distance	Meaning	Example
0	Exact prediction	14 winners
1	One total digit-step away	83 players
2	Two total digit-steps away	417 players
3+	Progressively farther	...

3.3 Two independent post-round stories

Voting story	Prediction story
Which digits won each D'Hondt position	Winning number
Vote totals and decisive quotients	Player prediction
Winning digit / position contribution	Digit distance
Runner-up quotient and margin	Prediction rank/tier

3.4 Player result card

ROUND #402
Winning number: 527

YOUR PREDICTION: 528
Digit distance: 1
Prediction tier: 1 away

YOUR VOTE: 2
Vote outcome: helped determine position 2

Vote outcome is feedback, not a hidden second scoring system. It never modifies the prediction histogram or winner set.

4. Persistence & Data Architecture

4.1 Source-of-truth rule

PostgreSQL is authoritative for all player state that can affect the outcome. Redis is never the sole durable store for a draft, committed vote, prediction, or result. If Redis disappears, the round remains correct and recoverable.

Data	Authoritative	Redis role
Draft prediction/vote	PostgreSQL	None required
Committed submission	PostgreSQL	Optional derived cache
Commit sequence	PostgreSQL	None
Final tally	PostgreSQL / deterministic derivation	Cache
Leaderboard	Derived from Postgres	Serving cache
Rate limits	—	Redis
Realtime fan-out	—	Redis/pub-sub if needed
Audit / commitment	PostgreSQL	None

4.2 Why not Redis-first?

A Redis-success response followed by asynchronous batching creates a correctness window: Redis can acknowledge a player while the batch has not yet reached PostgreSQL. If Redis then fails, the player believes the draft was saved but the authoritative record is gone. Therefore durable acknowledgement occurs only after PostgreSQL persistence.

4.3 Recommended schema

```
rounds(  
  round_id PK, cycle_number UNIQUE, status,  
  opened_at, blackout_at, locked_at, revealed_at,  
  ruleset_version, algorithm_version, commitment_root  
)  
  
round_drafts(  
  round_id, user_id,  
  prediction CHAR(3), vote SMALLINT,  
  version BIGINT, updated_at,  
  PRIMARY KEY(round_id, user_id)  
)  
  
submissions(  
  submission_id PK,  
  round_id, user_id,  
  prediction CHAR(3), vote SMALLINT,  
  committed_at, commit_sequence,  
  UNIQUE(round_id, user_id)  
)
```

Use fixed-width CHAR(3) or an integer plus canonical zero-padding; never allow ambiguous representations such as 7 versus 007.

5. Transactional Submission & Deadline Sealing

5.1 Manual lock-in

```
BEGIN
  verify round accepts submissions
  validate prediction + vote
  verify draft/version if applicable
  INSERT immutable submission
  allocate commit_sequence
  finalize/remove draft
COMMIT
```

→ only then return "LOCKED & VERIFIED"

The client receives success only after the authoritative transaction commits. A network failure after commit is handled through idempotent retry/status lookup; it must not create a duplicate submission.

5.2 Draft editing

Draft updates are durable PostgreSQL updates. Use optimistic versioning so two tabs/devices cannot silently overwrite one another. A conflicting version returns a refresh/retry response.

```
UPDATE round_drafts
SET prediction=?, vote=?, version=version+1, updated_at=NOW()
WHERE round_id=? AND user_id=? AND version=?;
```

5.3 Deadline sealing

```
At T+24:
  enter SEALING state
  reject new draft mutations / lock-in requests
  atomically finalize every valid remaining draft
  assign commit_sequence values
  finalize the authoritative submission set
  record commitment root
  mark round SEALED
```

Sealing must be idempotent and restartable. A worker crash halfway through must never produce duplicate submissions or an ambiguously sealed round. Prefer database constraints and transactional state transitions.

5.4 Ordering

commit_sequence is an authoritative database order for committed entries. committed_at remains audit metadata. Neither timestamp nor sequence affects prediction winner selection; sequence exists for traceability and future-compatible ordering semantics.

6. Lifecycle Orchestration, Concurrency & Failure Recovery

6.1 Round state machine

OPEN ■■■■■■■■■■ BLACKOUT ■■■■■■■■■■ SEALING ■■■■■■ SEALED ■■■■■■ REVEALED
T+0 T+23h T+24h after seal T+30h

The database, not an application timer alone, is authoritative for whether a transition is legal. Workers may trigger transitions, but every mutation re-checks round state.

6.2 Concurrency cases

Scenario	Required behavior
Two tabs edit draft	Optimistic version conflict; no silent overwrite.
Lock-in retried	Idempotency key / unique round+user constraint prevents duplicate.
Client times out after commit	Retry returns existing committed status.
Deadline races with edit	Sealing establishes authoritative cutoff; post-seal mutation rejected.
Worker crashes during sealing	Restart safely; constraints prevent duplicates.
Redis outage	Derived views may degrade; authoritative state remains intact.
Postgres unavailable	Do not acknowledge save/commit.

6.3 Idempotency

Manual lock-in requests should carry a client-generated idempotency key. Persist the key/result or use the unique submission constraint plus deterministic status lookup. The API must be safe to retry.

6.4 Time

Store UTC timestampz values. Round boundaries are computed from canonical times stored in the database. Client clocks never determine eligibility. Microsecond storage is useful for audit, but correctness comes from authoritative state transitions.

7. API, Frontend & Technology Stack

7.1 Recommended stack

Layer	V1 technology	Purpose
Web	Next.js 16+ / React / App Router	Game console, reveal, history
UI	Tailwind CSS + Framer Motion	Responsive console and reveal animation
API	FastAPI / Python 3.12+	Validation and transactions
Database	PostgreSQL 17	Durable source of truth
Cache	Redis 7.x	Leaderboard/cache, rate limiting, realtime
Workers	Temporal preferred; Celery acceptable	Materialization, sealing, resolution
Edge	Cloudflare + TLS + Turnstile	Protection and bot friction
Observability	Structured logs + metrics + traces	Latency, errors, integrity

7.2 Core API surface

```
GET /rounds/current
GET /rounds/{id}
GET /rounds/{id}/leaderboard
GET /rounds/{id}/my-entry

PUT /rounds/{id}/draft
POST /rounds/{id}/lock
GET /rounds/{id}/submission/status

GET /rounds/{id}/result
GET /rounds/{id}/audit
```

The API validates all values server-side. prediction must be exactly three decimal digits; vote must be an integer 0–9; round and user authorization are mandatory.

7.3 Leaderboard caching

Hourly public metrics are materialized from PostgreSQL into Redis. During blackout, stop serving live tallies and expose only the permitted frozen/historical view. Cache invalidation cannot affect final resolution.

7.4 Frontend states

OPEN: editable draft and visible metrics. BLACKOUT: editable draft but live metrics hidden. LOCKED: immutable personal entry. SEALED/RESOLUTION: no mutations. REVEALED: final result and audit UI.

8. Security, Integrity & Auditability

8.1 V1 threat model

Threat	V1 control
Bot submissions	Turnstile + rate limits + behavioral detection
Mass account abuse	Account controls + basic device/network heuristics + anomaly detection
Vote tampering	Immutable Postgres submission + sealed round state
Duplicate entry	Unique round_id + user_id constraint
Client manipulation	Server-side validation and authoritative state
Redis loss	No authoritative state depends on Redis
Post-lock mutation	Database state transition rejects writes
Algorithm drift	ruleset_version + algorithm_version
Post-hoc manipulation	Immutable input set + cryptographic commitment + deterministic resolver

8.2 Coordination policy

Social coordination is intended gameplay. V1 does not attempt to prevent people from communicating externally or organizing votes. The anti-abuse boundary is automated/fraudulent participation and technical circumvention—not strategic coalition behavior.

8.3 Cryptographic commitment

At sealing, create a canonical ordered representation of the immutable submission set and compute a cryptographic commitment, preferably a Merkle root. Canonicalization must specify field order, encoding, zero-padding, ordering key and algorithm version.

```
canonical_submission =  
    round_id | submission identifier | prediction |  
    vote | commit_sequence
```

```
leaf_i = Hash(canonical_submission_i)  
root   = MerkleRoot(leaves in commit order)
```

At reveal, publish the root and sufficient audit material for independent reconstruction. The original salt commitment may be retained as an additional anti-post-hoc mechanism, but it is not by itself proof that the submission dataset was unchanged.

8.4 Audit ledger

Record each D'Hondt slot, winning digit, winning quotient numerator/denominator, runner-up digit and quotient, resolution timestamp, ruleset version and input commitment root. Audit data is append-only after sealing.

9. Operations, Testing & Implementation Checklist

9.1 Performance target

The target is up to 500,000 concurrent submissions in the final 10 minutes without unacceptable latency. Treat this as a load-test target, not a reason to make Redis authoritative. Benchmark draft writes, lock-in transactions, deadline sealing, leaderboard reads and resolution separately.

9.2 Mandatory test suites

Suite	Must prove
Math/property	D'Hondt deterministic; rational comparison has no floating-point drift.
Prediction	Digit distance handles leading zeroes and all 000-999 values.
Winner	All minimum-distance players win; prediction frequency never changes outcome.
Deadline	Last-moment edit vs sealing produces one deterministic state.
Idempotency	Retries cannot duplicate or alter a committed submission.
Failure	Redis loss, worker crash, API timeout and DB failover do not lose committed state.
Concurrency	Two tabs/devices and high-volume bursts remain correct.
Audit	Replayed immutable inputs reproduce number, histogram and winners.
Security	Unauthorized mutation, bot bursts and API abuse are appropriately rejected/detected.

9.3 Implementation order

- Phase 1: PostgreSQL schema, round state machine, durable drafts and submission transaction.
- Phase 2: D'Hondt reference engine + exhaustive/property tests + digit-distance resolver.
- Phase 3: FastAPI contract, idempotency, deadline sealing and recovery behavior.
- Phase 4: Next.js console, blackout UX, reveal/histogram and personal result.
- Phase 5: Redis leaderboard/rate-limit acceleration and worker orchestration.
- Phase 6: Cryptographic commitment/audit tooling, observability, security controls and load testing.

9.4 V1 acceptance criteria

- No acknowledged player draft or committed submission can be lost because Redis, an API worker or a background worker failed.
- Every round produces the same winning number from the same committed vote set.
- Zero votes produce 000 through ordinary D'Hondt + permanent lower-digit tie resolution.
- Every player at the minimum digit distance is a winner; timestamps never eliminate equal predictions.
- Prediction histogram is computed only after the winning number is fixed and cannot influence it.
- Manual and automatic lock-in converge on the same immutable submission model.
- A sealed round cannot be mutated through any API path.
- An independent resolver can reproduce the winning number, prediction distances and winner set from the committed dataset.

FINAL V1 PRINCIPLE

Build the game so that strategy is flexible, state is durable, outcomes are deterministic, and rules are understandable. Do not over-engineer identity into an election system; do not under-engineer persistence into a cache. PostgreSQL defines what happened. The game rules define who won.